

Air University



Digital Logic Design (Lab)

Year 2025

Ms. Faryal Naeem

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: March 08, 2025

Experiment No. 04

Title: Boolean Expressions – Simplification and Implementation

Objectives:

- Understand the basics of Boolean algebra and its significance in simplifying complex Boolean expressions.
- Develop the skills needed to simplify Boolean expressions using algebraic methods.
- Implement simplified Boolean expressions through the use of AND, OR, and NOT logic gates.
- Verify the accuracy of the simplified expressions by creating truth tables.

Equipment and Components:

- Proteus Software (for simulation purposes)
- Logic Trainer Kit (for hands-on experience)
- Logic Gate Integrated Circuits (7408 - AND, 7432 - OR, 7404 - NOT)
- Breadboard
- Power Supply (5V)
- Connecting Wires
- Multimeter (optional)

Introduction:

Boolean algebra serves as a powerful tool for analyzing and simplifying digital circuits. Reducing complex Boolean expressions is essential in digital electronics because it minimizes the number of logic gates required, resulting in systems that are more efficient, cost-effective, and reliable. This experiment focuses on simplifying two specific Boolean expressions:

- i. $A + AB + A \bar{B} C$
- ii. $(\bar{A} + B) C + ABC$

We will apply algebraic manipulation techniques to simplify these expressions, using Boolean identities such as absorption law and distributive law among others. The simplified expressions will then be implemented using basic logic gates: AND, OR, and NOT. We will also confirm the functionality and equivalence of both the original and simplified expressions through truth table analysis and simulations in Proteus software. This process showcases the practical application of Boolean algebra in optimizing digital circuit design.

Procedure:

Using Proteus Software:

1. **Open Proteus:** Launch the Proteus software and create a new project.
2. **Design a Circuit:** Build a logic circuit according to the initial Boolean expression.
3. **Simplify the Expression:** Apply Boolean identities to algebraically simplify the expression.
4. **Implement the Simplified Expression:** Use logic gates to realize the simplified expression.

5. **Simulate:** Run simulations for both the original and simplified circuits to ensure they function equivalently.

Using Logic Trainer:

1. **Set up the Breadboard:** Connect the power supply to the breadboard.
2. **Insert Logic Gate ICs:** Place the necessary logic gate ICs (AND, OR, NOT) onto the breadboard.
3. **Implement the Simplified Expression:** Wire the gates according to the simplified Boolean expression.
4. **Test Input Combinations:** Methodically test each possible input combination and record the outputs using either a multimeter or LED indicators.
5. **Construct Truth Tables:** Create truth tables for both the original and simplified expressions to confirm their equivalence.

Simplification Techniques:

Algebraic Simplification:

Leverage Boolean identities including:

- Commutative Law
- Associative Law
- Distributive Law
- Absorption Law
- DeMorgan's Laws

Utilize basic rules like:

- $A + 0 = A$
- $A + 1 = 1$
- $A \cdot 0 = 0$
- $A \cdot 1 = A$
- $A + A = A$
- $A \cdot A = A$
- $A + \bar{A} = 1$
- $A \cdot \bar{A} = 0$

Example Simplification and Implementation:

Let's consider the following Boolean expressions:

- i. $A + AB + A \bar{B} C$
- ii. $(\bar{A} + B) C + ABC$

We will simplify these expressions using Boolean algebra, implement them using AND, OR, and NOT gates, and then verify their functionality using truth tables.

Logic Expressions and Truth Tables:

- i. $A + AB + A \bar{B} C$

Algebraic Simplification:

- Use the absorption law ($A + AB = A$): $A + A \bar{B} C$
- Apply the absorption law again: $A + AX = A$ (where X is any term: $\bar{B} C = X$)
- Therefore, the final simplified expression is: A

Implementation:

1. Use an AND gate to perform AB .

2. Use a NOT gate to get $\bar{B}$.
3. Use an AND gate to perform $A \bar{B}$.
4. Use an AND gate to perform $A \bar{B}C$.
5. Use an OR gate to combine A and AB .
6. Use an OR gate to combine the result of step 5 and $A \bar{B}C$.

Truth Table:

A	B	C	AB	$\bar{B}$	$A \bar{B}$	$A \bar{B}C$	$A+AB$	$A+AB+A \bar{B}C$
0	0	0	0	1	0	0	0	0
0	0	1	0	1	0	0	0	0
0	1	0	0	0	0	0	0	0
0	1	1	0	0	0	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	0	1	1	1	1	1
1	1	0	1	0	0	0	1	1
1	1	1	1	0	0	0	1	1

ii. $(\bar{A} + B)C + ABC$

Algebraic Simplification:

- $\bar{A}C + BC + ABC$
- $\bar{A}C + C(B + AB)$
- Apply absorption law ($B + AB = B$): $\bar{A}C + BC$
- $\bar{A}C + BC$

Implementation:

1. Use a NOT gate to get $\bar{A}$.
2. Use an OR gate to combine $\bar{A}$ and B : $\bar{A} + B$.
3. Use an AND gate to combine $\bar{A} + B$ and C : $(\bar{A} + B)C$.
4. Use an AND gate to combine A and B : AB .
5. Use an AND gate to combine AB and C : ABC .
6. Use an OR gate to combine results from step 4 And step 6: $(\bar{A} + B)C + ABC$.

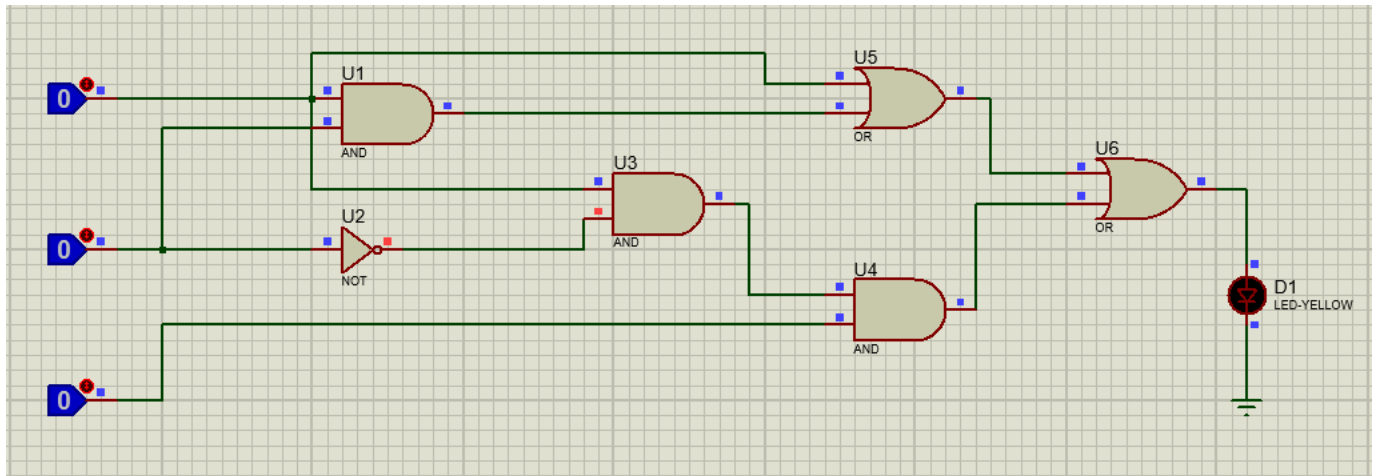
Truth Table:

A	B	C	$\bar{A}$	$\bar{A}C$	BC	$\bar{A}C + BC$	$(\bar{A} + B)C + ABC$
0	0	0	1	0	0	0	0
0	0	1	1	1	0	1	1
0	1	0	1	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0
1	1	0	0	0	0	0	0
1	1	1	0	1	1	1	1

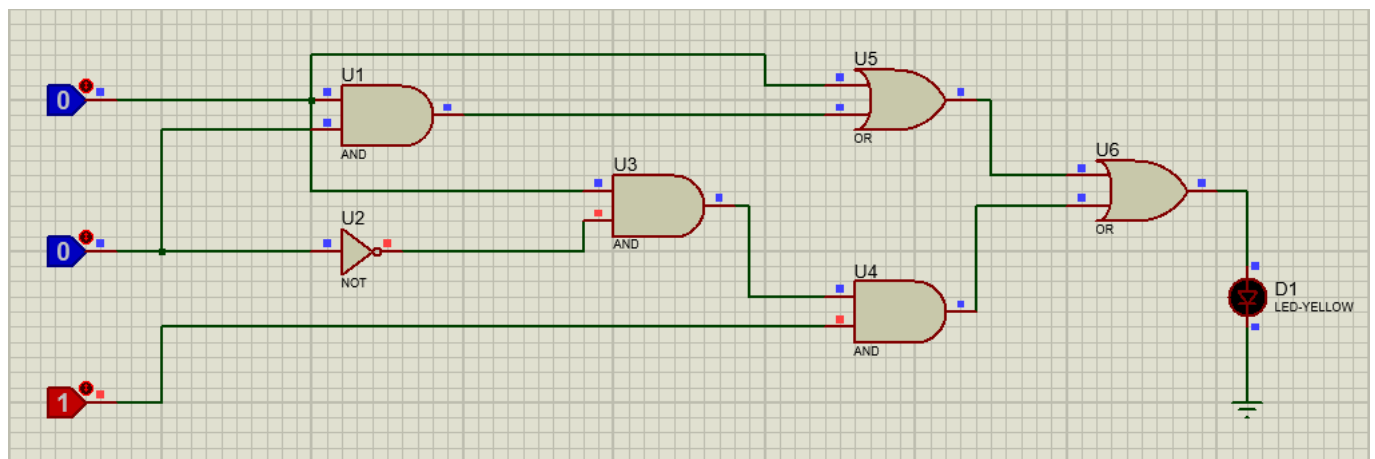
Simulated Results:

i. $A + AB + A \bar{B} C$

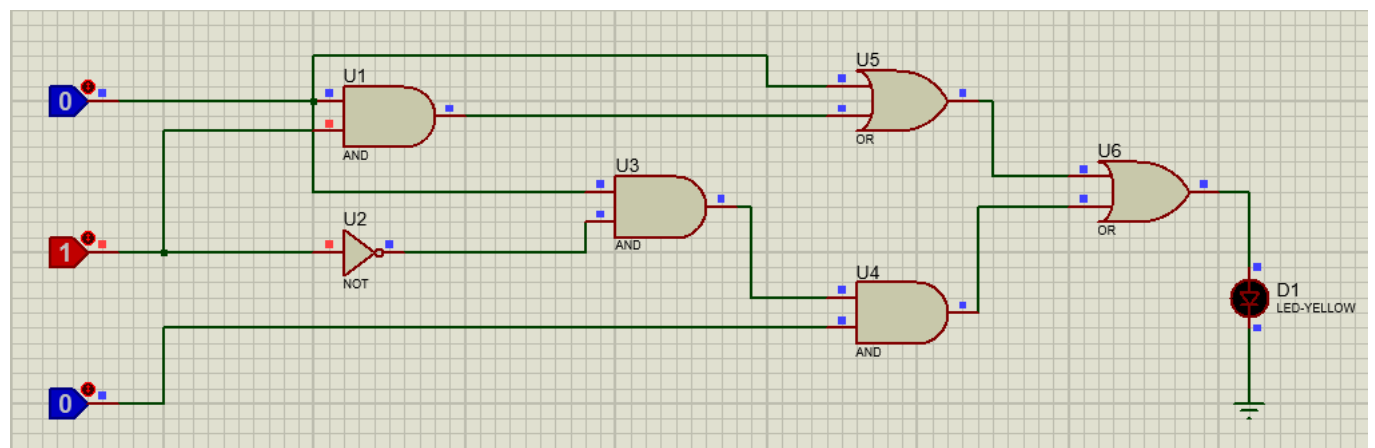
Input: A=0, B=0, C=0 → Output: 0



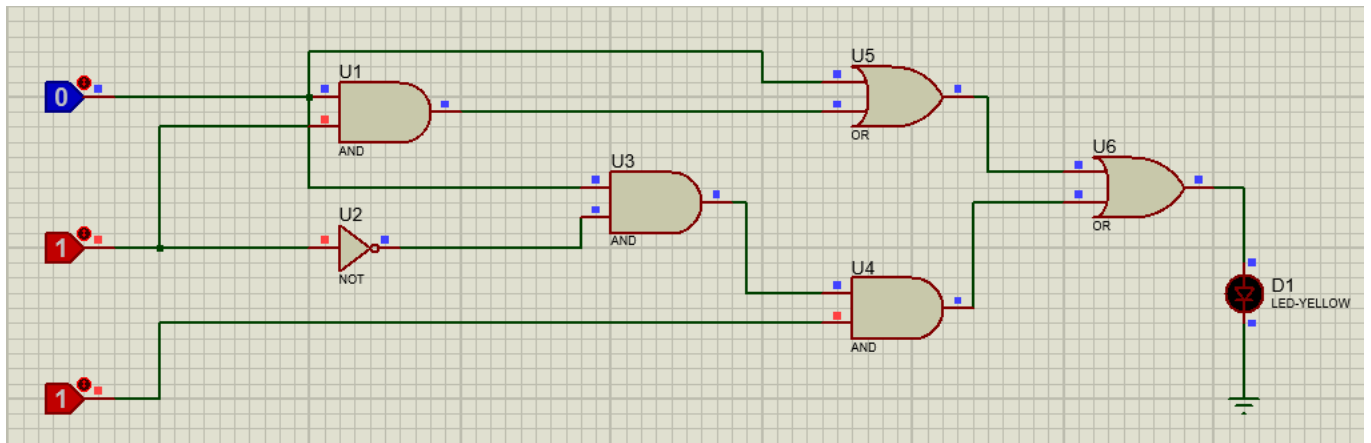
Input: A=0, B=0, C=1 → Output: 0



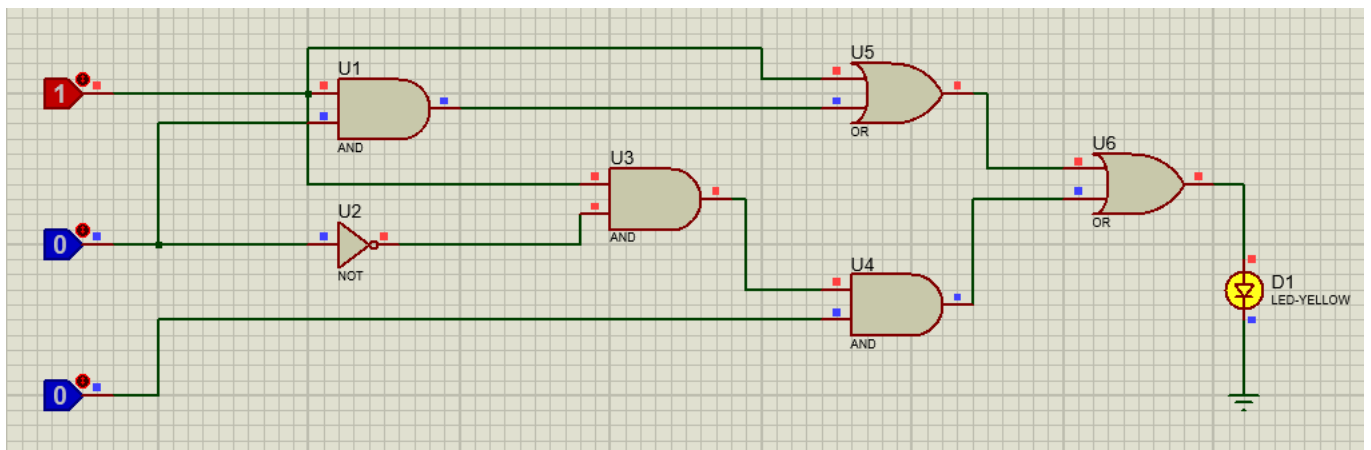
Input: A=0, B=1, C=0 → Output: 0



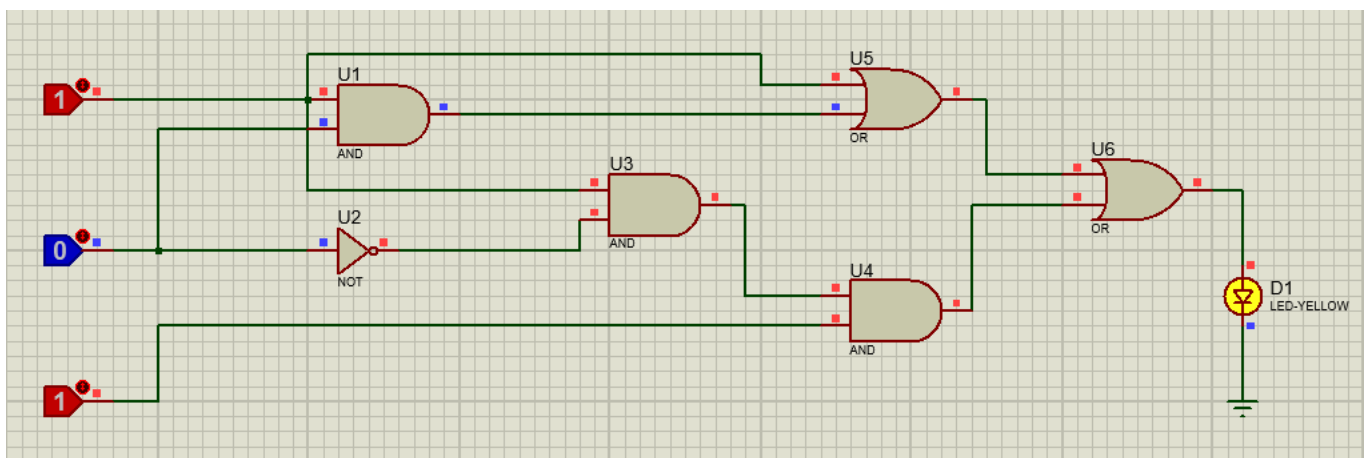
Input: A=0, B=1, C=1 → Output: 0



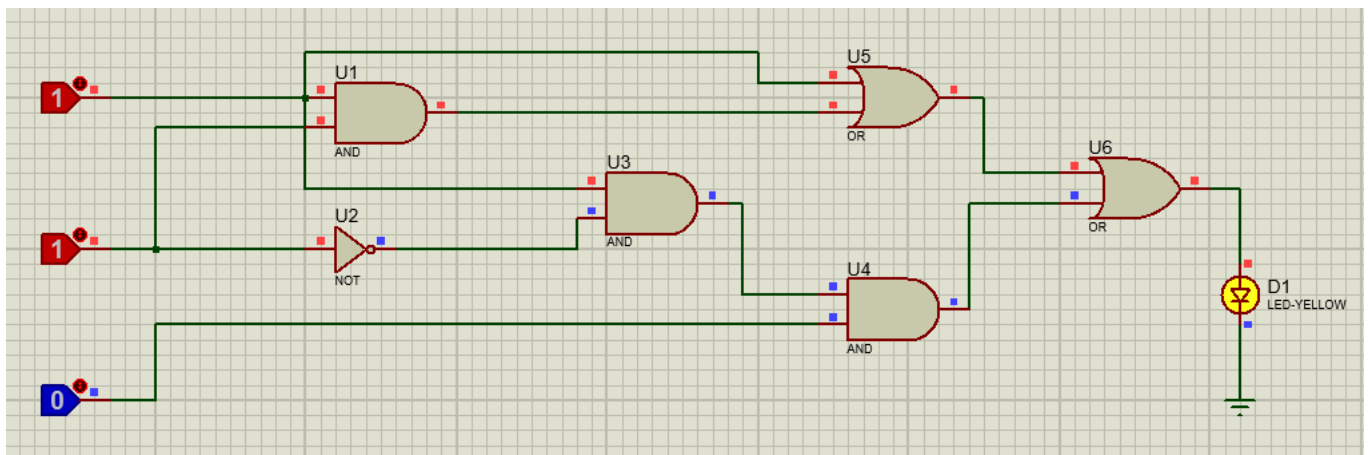
Input: A=1, B=0, C=0 → Output: 1



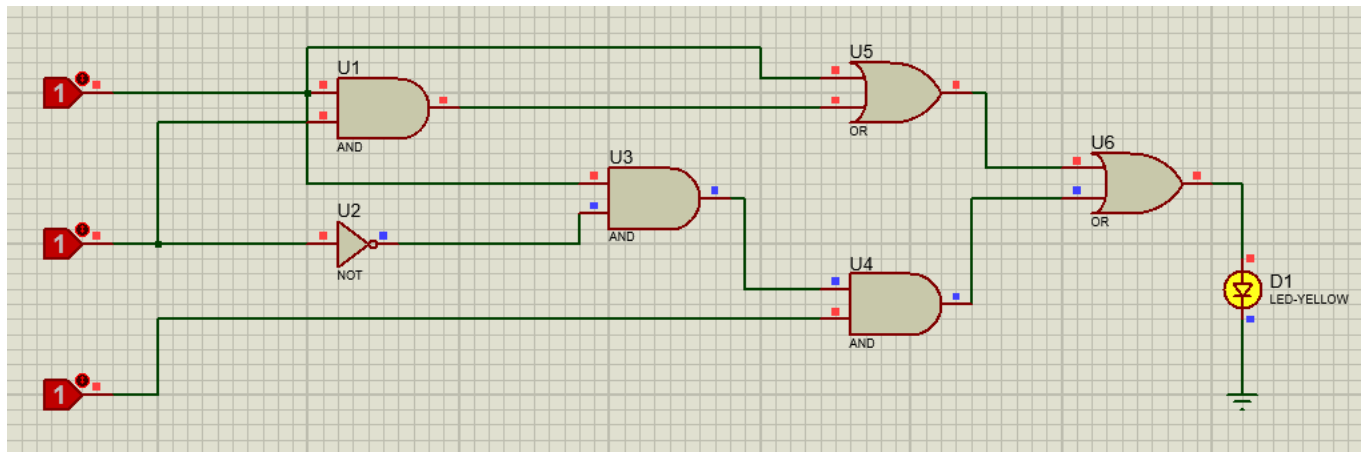
Input: A=1, B=0, C=1 → Output: 1



Input: A=1, B=1, C=0 → Output: 1

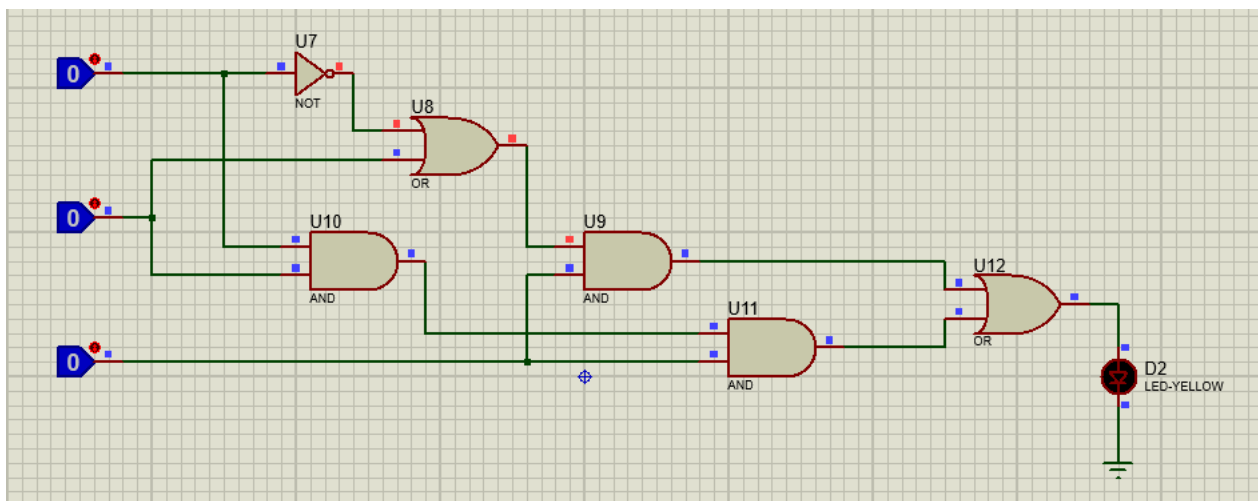


Input: A=1, B=1, C=1 → Output: 1

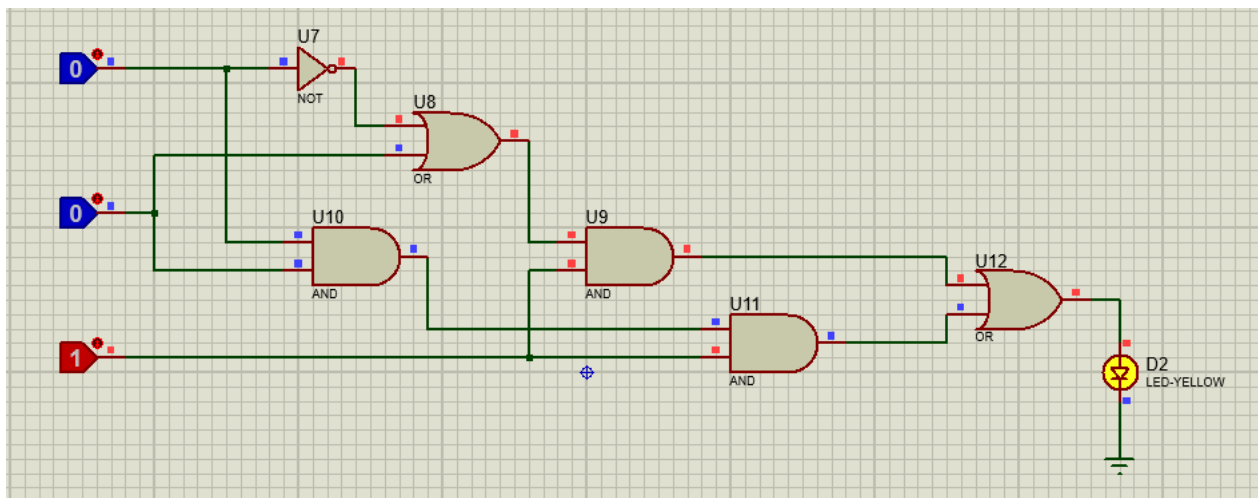


ii. $(\bar{A} + B)C + ABC$

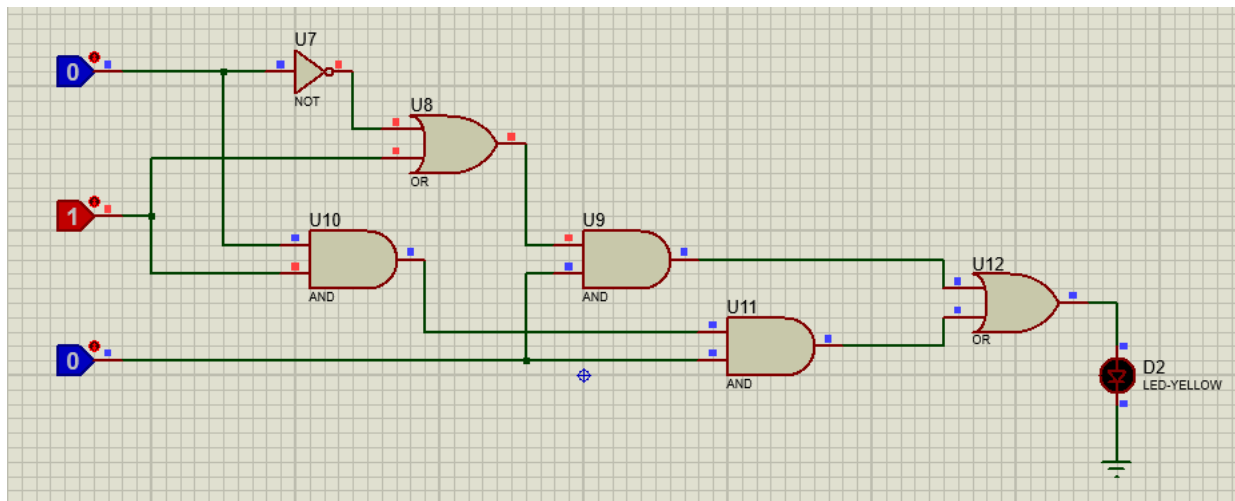
Input: A=0, B=0, C=0 → Output: 0



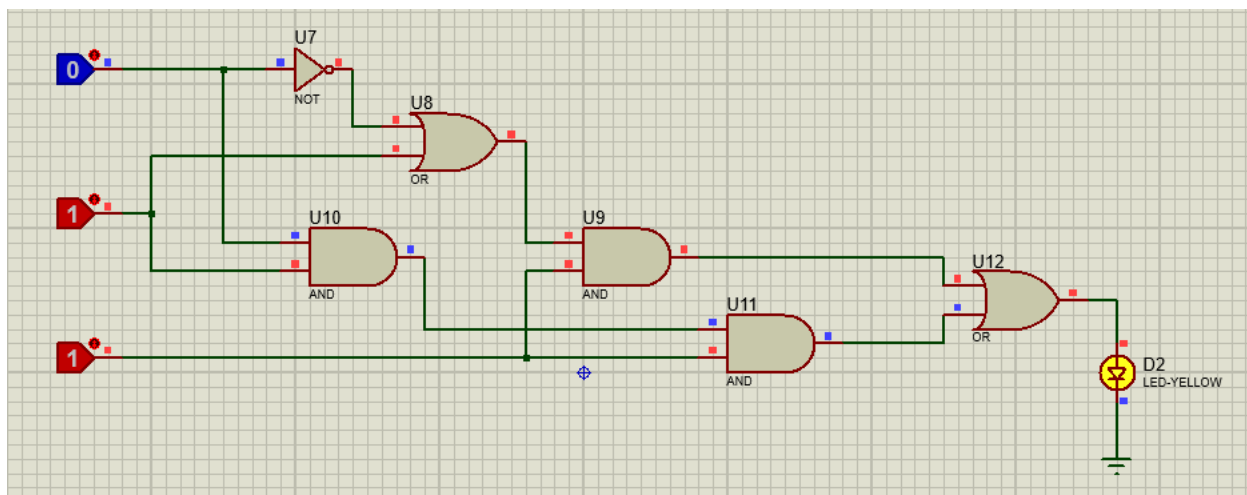
Input: A=0, B=0, C=1 → Output: 1



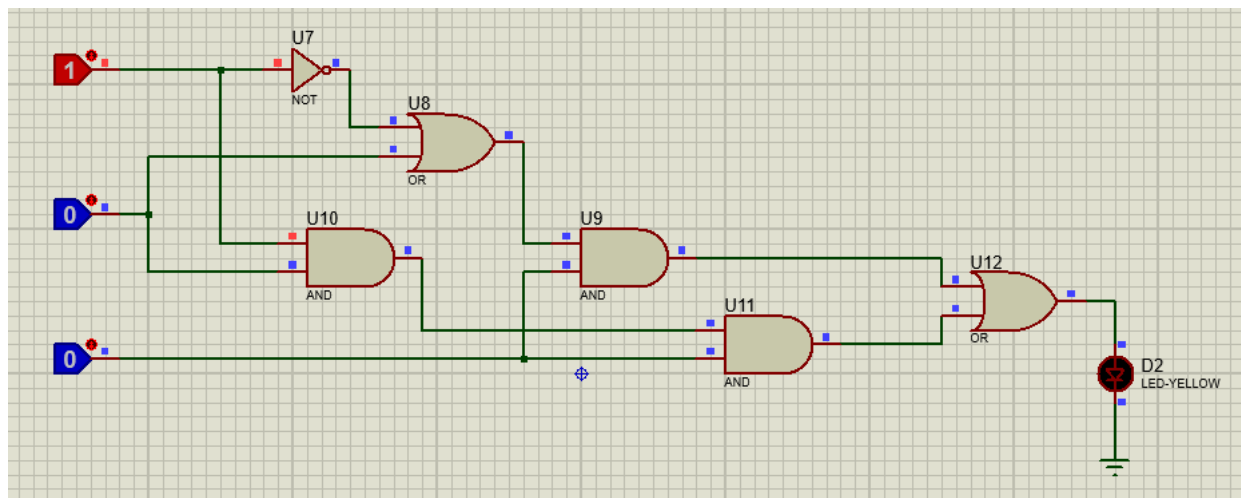
Input: A=0, B=1, C=0 → Output: 0



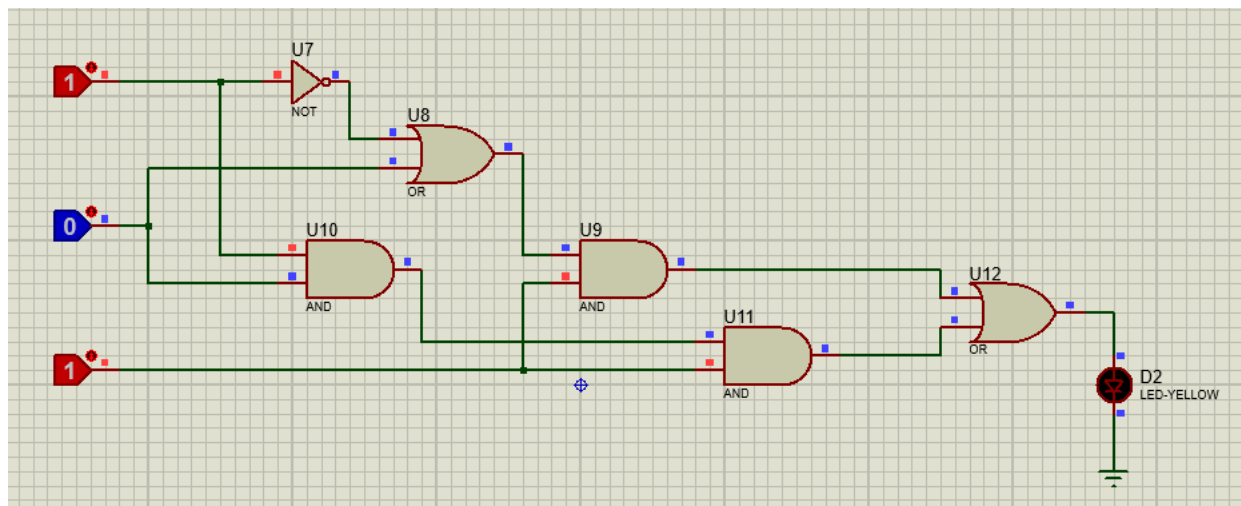
Input: A=0, B=1, C=1 → Output: 1



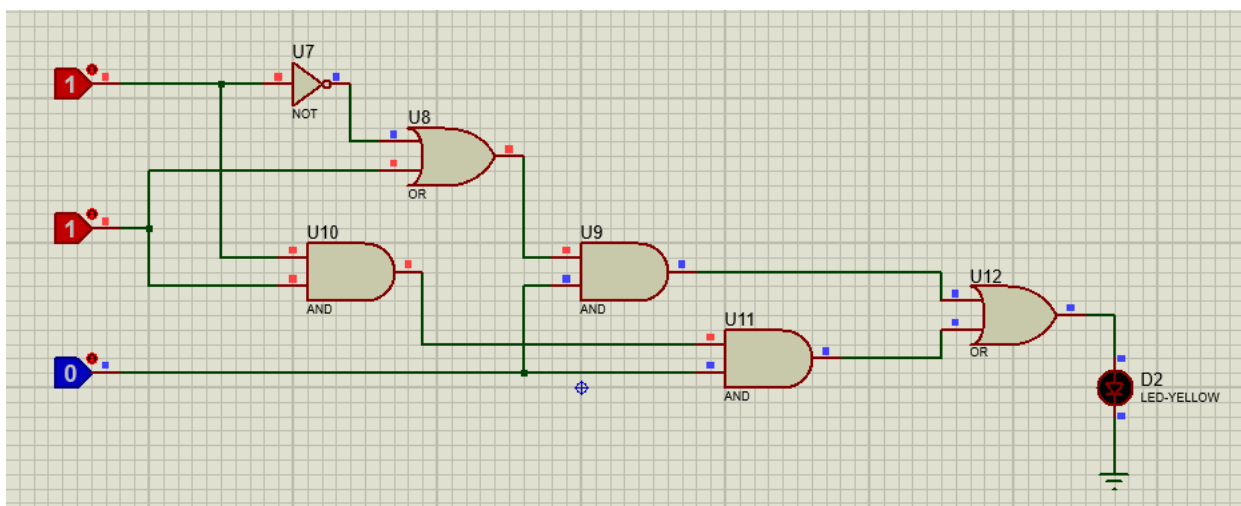
Input: A=1, B=0, C=0 → Output: 1



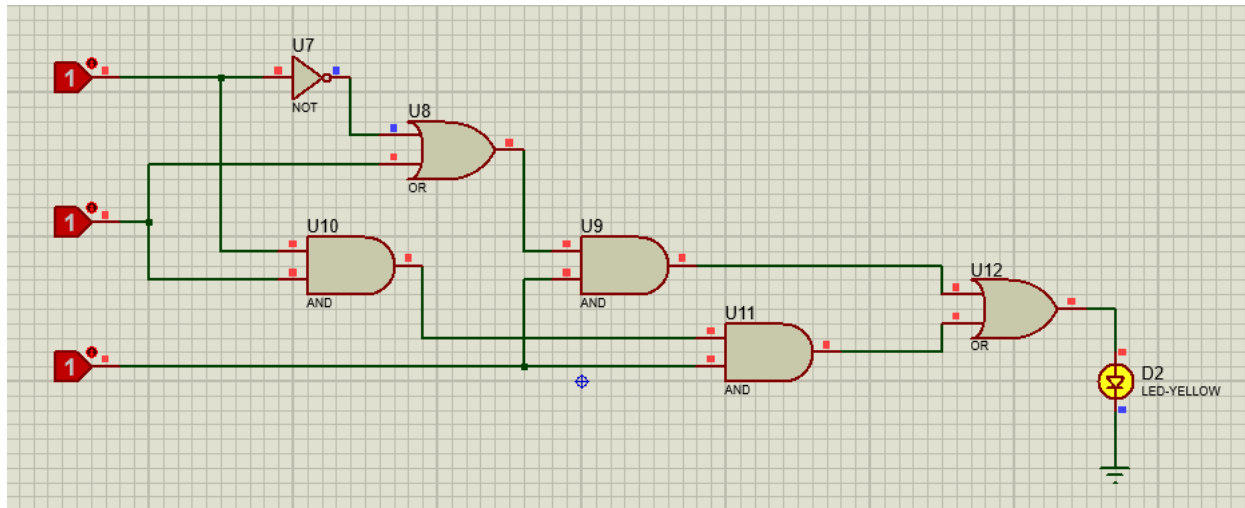
Input: A=1, B=0, C=1 → Output: 0



Input: A=1, B=1, C=0 → Output: 0



Input: A=1, B=1, C=1 → Output: 1



Conclusion:

This experiment effectively illustrated the process of simplifying Boolean expressions and implementing them with logic gates. Through algebraic manipulation and truth table validation, we demonstrated that complex expressions can be transformed into simpler, equivalent forms. This simplification is crucial in digital circuit design, promoting systems that are not only efficient and cost-effective but also more reliable. Mastering the ability to optimize circuits by reducing the number of logic gates is a fundamental skill in digital electronics and computer science, paving the way for the design and analysis of increasingly complex digital systems. Future explorations could include the simplification of more intricate multi-variable Boolean functions and comparing them with software-based optimization tools for large-scale digital design.

THANK YOU FOR YOUR TIME!

THE END